

CST-407 Activity 5 Spring Boot Security Configuration

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/19/2025

Table of Contents

Part 1 — Adding Security to a Spring Boot Application.....	4
1.1 Starting Project.....	4
Database Config Updated for MAMP.....	4
Unsecured Orders Page (Before Security).....	5
1.2 Add Dependencies.....	6
Security Dependency Added	6
Default Spring Security Login Page.....	7
1.3 Create the Security Configuration Class	8
Security Package Created	8
SecurityConfig Class Implemented	9
1.4 Password Encoding Configuration.....	10
Password Encoder Bean (BCrypt).....	10
1.5 Update the User Service for Authentication.....	11
UserDetailsService Implemented (DB-backed)	11
Part 2 – Custom Login Page.....	12
2.1 – Custom Login HTML.....	12
Custom Login HTML	12
2.2 – Update SecurityConfig to Use Custom Login	13
SecurityConfig Custom Login Update.....	13
2.3 – Test the Custom Login Page	14
Login Page (Custom) Redirect	14
2.4 – Verify Access After Login	15
Orders Page After Login	15
Logout Implemented as POST	16
Logout and Redirect to Home	17
Part 3: Role-Based Access Control (RBAC).....	18
3.1 – Add Roles to the User Entity.....	18
RBAC Rules in SecurityConfig	18
3.2 – Verify USER is denied on admin routes	19
USER Blocked from Admin Route.....	19
3.3 – Verify ADMIN can access admin routes.....	20
Admin Can Access Create Order	20
3.4 – Show/Hide Navbar Items Based on Login & Role	21
Thymeleaf Security Extras Dependency	21
Navbar (Logged Out)	22
Navbar (Admin — Create Visible).....	23
3.5 – Add Method-Level Security (defense in depth)	24
Enable Method-Level Security	24
PreAuthorize on OrdersController (Admin Only)	25
3.6 – Hide admin actions in the UI (Edit/Delete/Create link)	26

Orders Page Hides Admin Actions for USER.....	26
Orders Page Shows Admin Actions	27
3.7 – Add a Custom “Access Denied” (403) Page	28
Custom Access Denied Page.....	28
<i>Part 4 – Summary of Key Concepts</i>	<i>29</i>

Part 1 — Adding Security to a Spring Boot Application

1.1 Starting Project

Database Config Updated for MAMP

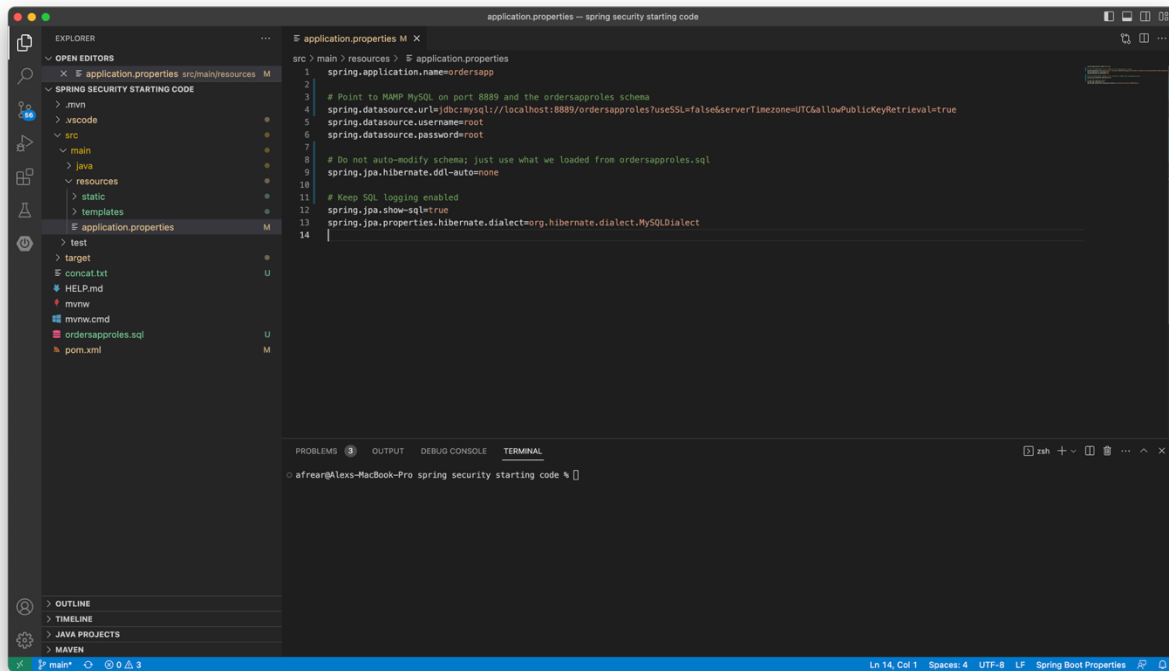


Figure 1 *application.properties* updated to use port 8889 and *ddl-auto=none*, matching the *ordersapproles* schema in MySQL Workbench.

Unsecured Orders Page (Before Security)

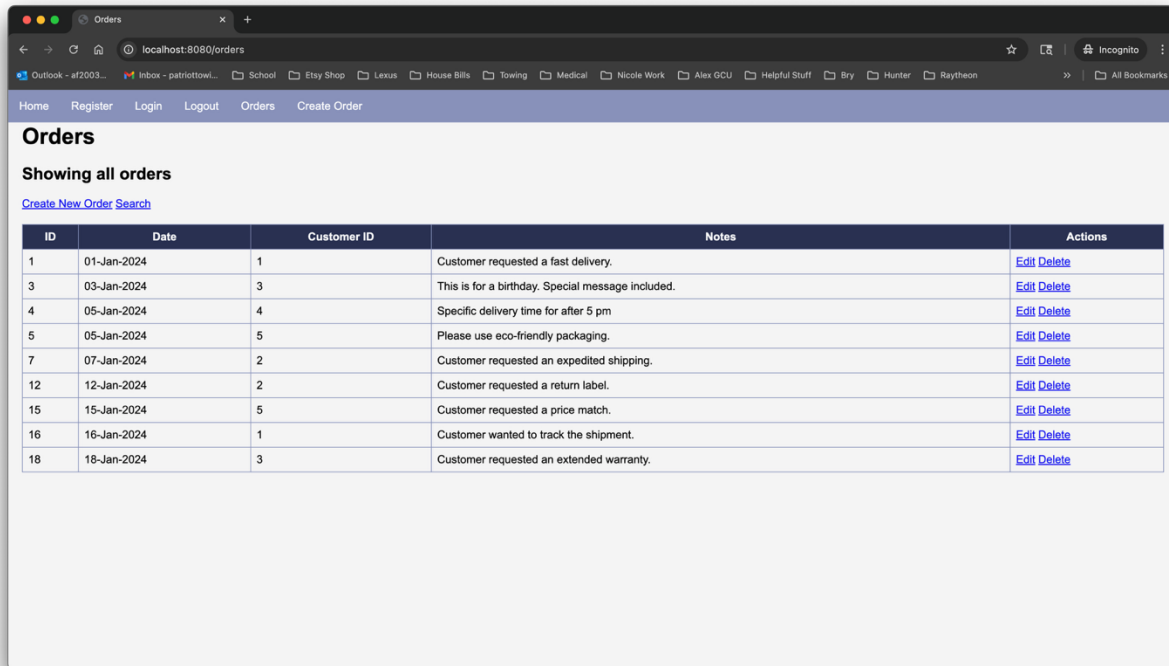


Figure 2 Orders page allows CRUD with no login, confirming security hasn't been applied yet.

1.2 Add Dependencies

Security Dependency Added

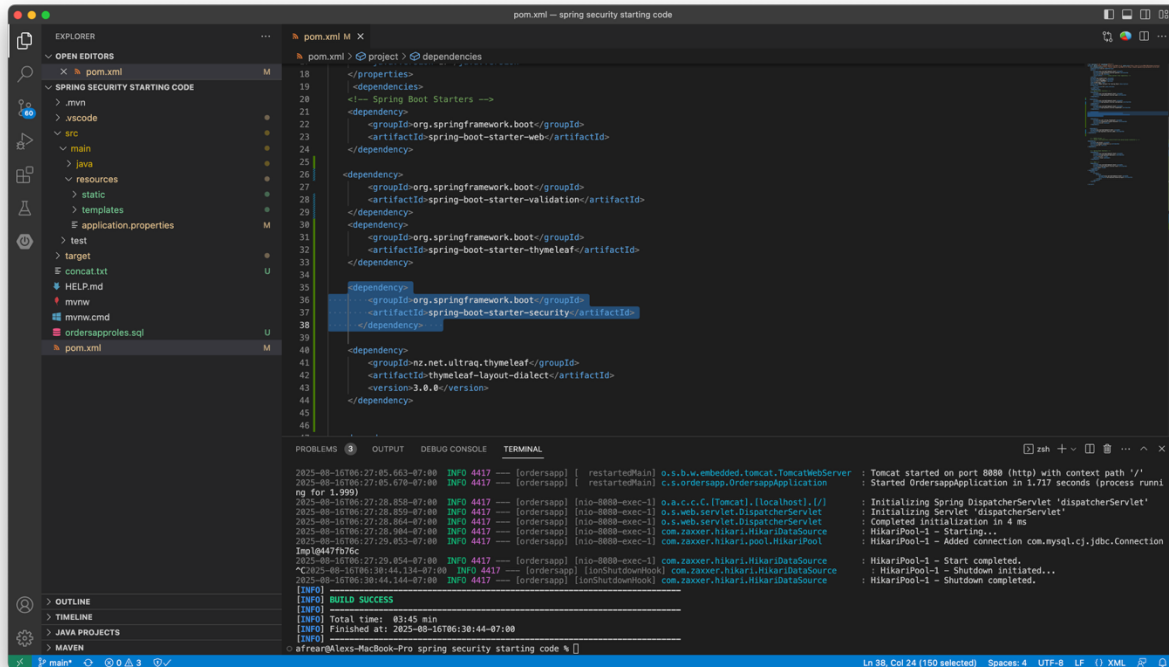


Figure 3 spring-boot-starter-security added to pom.xml, enabling Spring Security for the application.

Default Spring Security Login Page

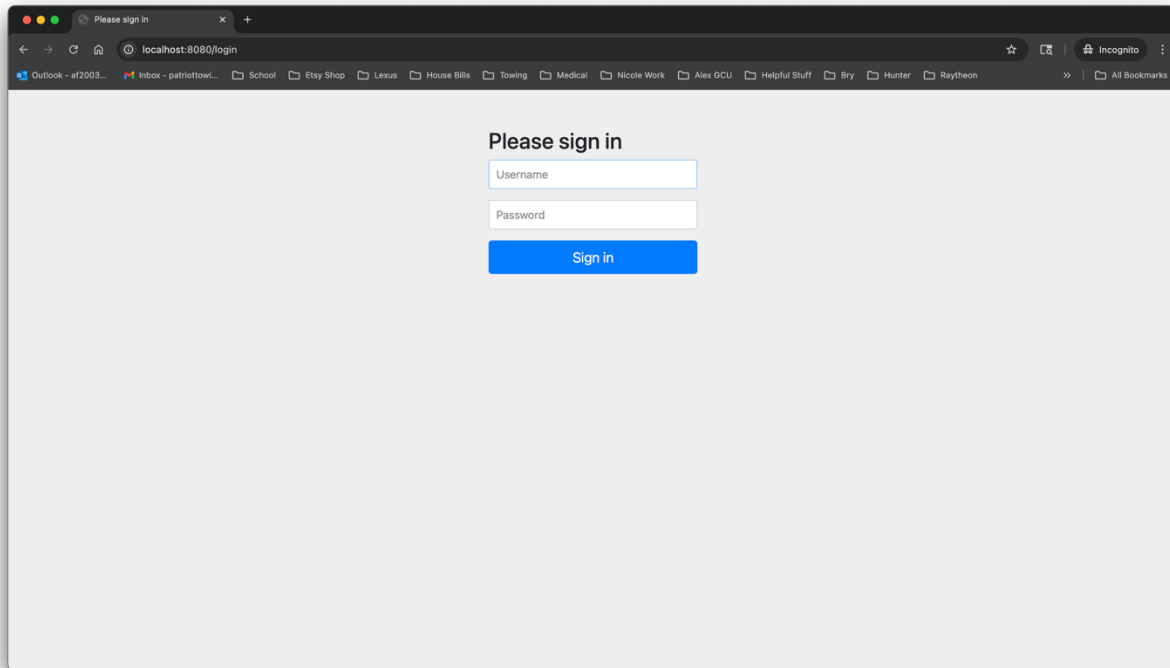


Figure 4 Default login page appears after adding the security starter, confirming Spring Security is active.

1.3 Create the Security Configuration Class

Security Package Created

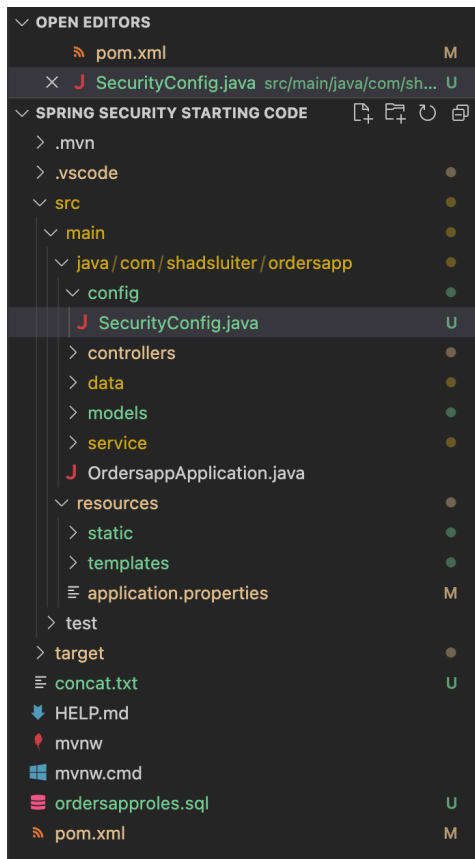
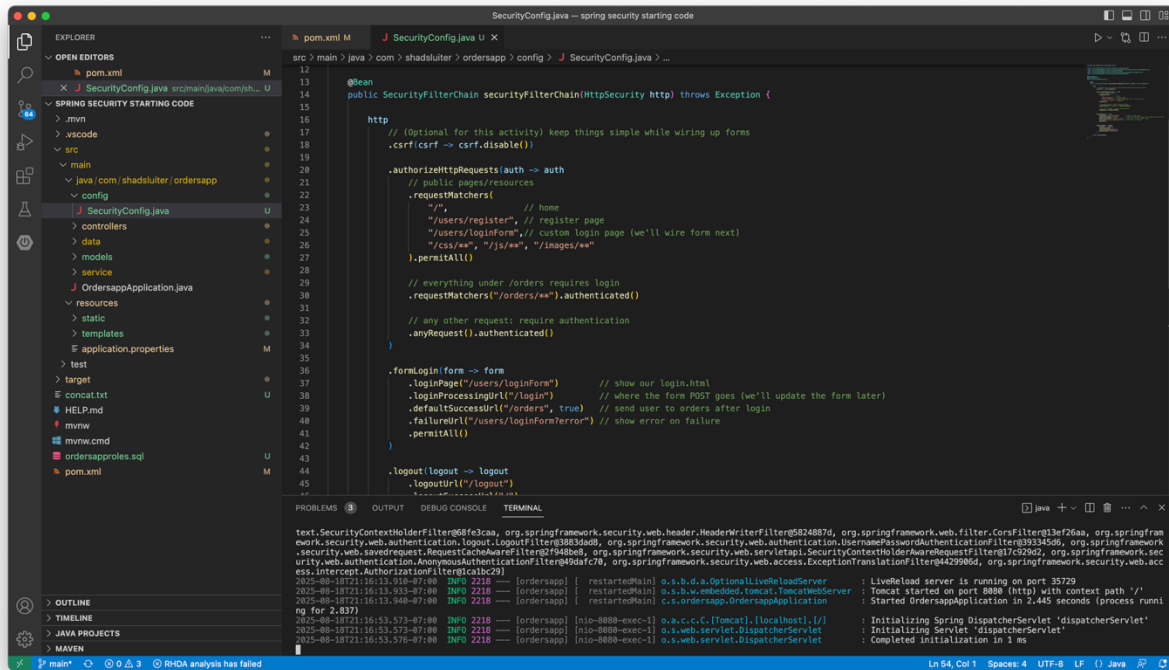


Figure 5 `config/` package created to hold Spring Security configuration classes.

SecurityConfig Class Implemented



The screenshot shows an IDE with the `SecurityConfig.java` file open. The file implements the `SecurityFilterChain` for `HttpSecurity`. The configuration includes:

- Disabling CSRF: `.csrf(csrf -> csrf.disable())`
- Authorizing requests to `/auth` with public pages/resources: `.authorizeHttpRequests(auth -> auth .requestMatchers( // public pages/resources "/", // home "/users/register", // register page "/users/loginForm", // custom login page (we'll wire form next) "/css/**", "/js/**", "/images/**" ).permitAll())`
- Requiring authentication for everything under `/orders`: `.requestMatchers("/orders/**").authenticated()`
- Requiring authentication for any other request: `.anyRequest().authenticated()`
- Configuring login: `.formLogin(form -> form .loginPage("/users/loginForm") // show our login.html .loginProcessingUrl("/login") // where the form POST goes (we'll update the form later) .defaultSuccessUrl("/orders", true) // send user to orders after login .failureUrl("/users/loginFormError") // show error on failure).permitAll())`
- Configuring logout: `.logout(logout -> logout .logoutUrl("/logout"))`

The terminal output shows the application starting successfully on port 8080:

```
text.SecurityContextHolderFilter@8f3caa, org.springframework.security.web.header.HeaderWriterFilter@5024887d, org.springframework.security.web.filter.CorsFilter@13ef26aa, org.springframework.security.web.authentication.logout.LogoutFilter@8883d4d8, org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter@33345d8, org.springframework.security.web.savedrequest.RequestCacheAwareFilter@f948b48, org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter@1c5295d, org.springframework.security.web.authentication.AnonymousAuthenticationFilter@49dafc7b, org.springframework.security.web.access.ExceptionTranslationFilter@4429986d, org.springframework.security.web.access.intercept.AuthorizationFilter@ca3bc29)
[ordersapp] | restartedMain | o.s.b.d.a.OptionalLiveReloadServer | LiveReload server is running on port 35729
[ordersapp] | restartedMain | o.s.b.w.embedded.tomcat.TomcatWebServer | Tomcat started on port 8080 (http) with context path '/'
[ordersapp] | restartedMain | c.s.ordersapp.OrdersappApplication | Started OrdersappApplication in 2.445 seconds (process running for 2.837)
[ordersapp] | inio-8080-exec-1 | o.a.c.c.C.[Tomcat].[localhost].[/] | Initializing Spring DispatcherServlet 'dispatcherServlet'
[ordersapp] | inio-8080-exec-1 | o.s.web.servlet.DispatcherServlet | Initializing Servlet 'dispatcherServlet'
[ordersapp] | inio-8080-exec-1 | o.s.web.servlet.DispatcherServlet | Completed initialization in 1 ms
```

Figure 6 `SecurityConfig` defines the `SecurityFilterChain` with public routes, authenticated `/orders/**`, custom login page, and logout behavior.

1.4 Password Encoding Configuration

Password Encoder Bean (BCrypt)

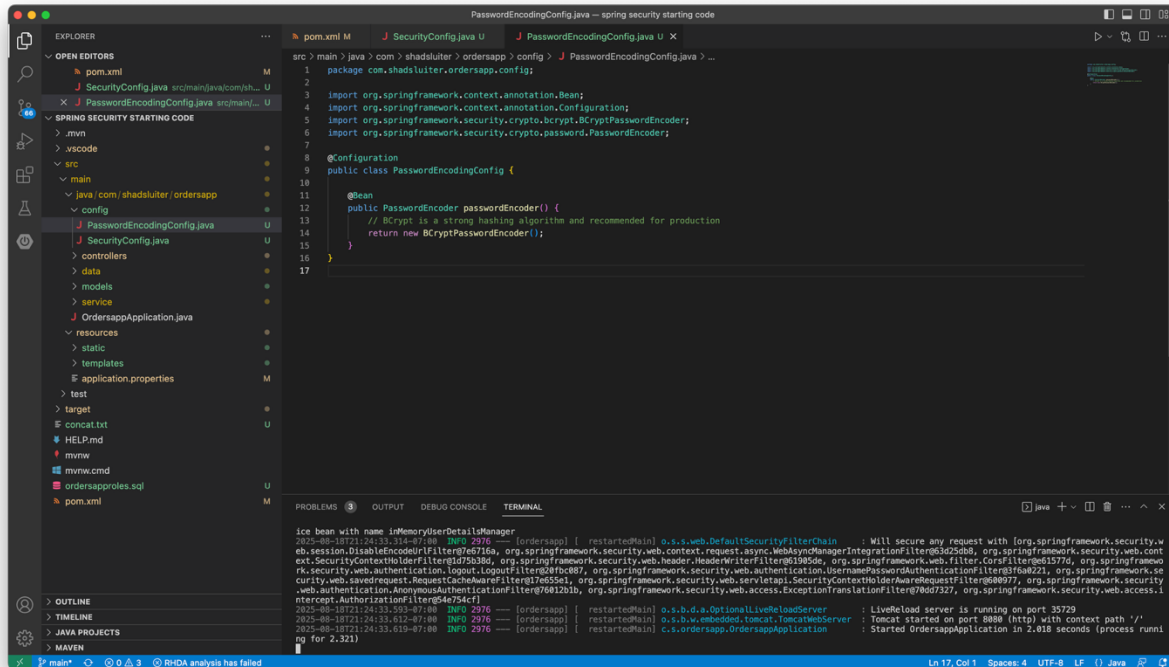


Figure 7 Bcrypt PasswordEncoder bean defined; recommended for securely hashing user passwords.

1.5 Update the User Service for Authentication

UserDetailsService Implemented (DB-backed)

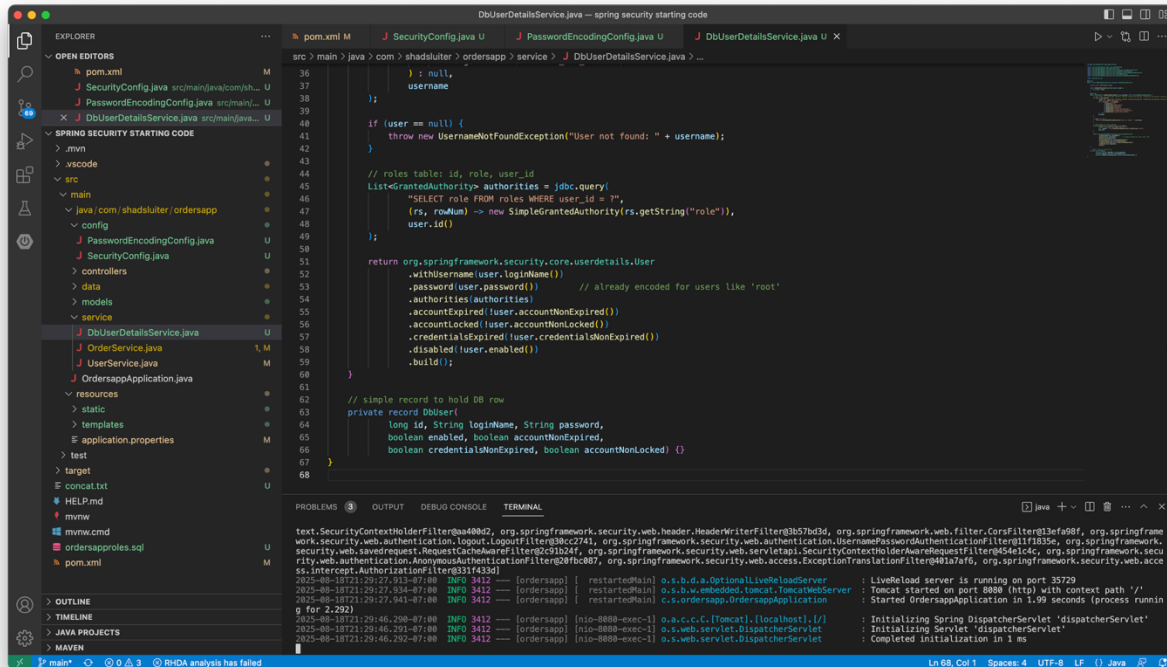


Figure 8 DbUserDetailsService loads users from users and roles from roles, returning a Spring Security User with authorities.

Part 2 – Custom Login Page

2.1 – Custom Login HTML

Custom Login HTML

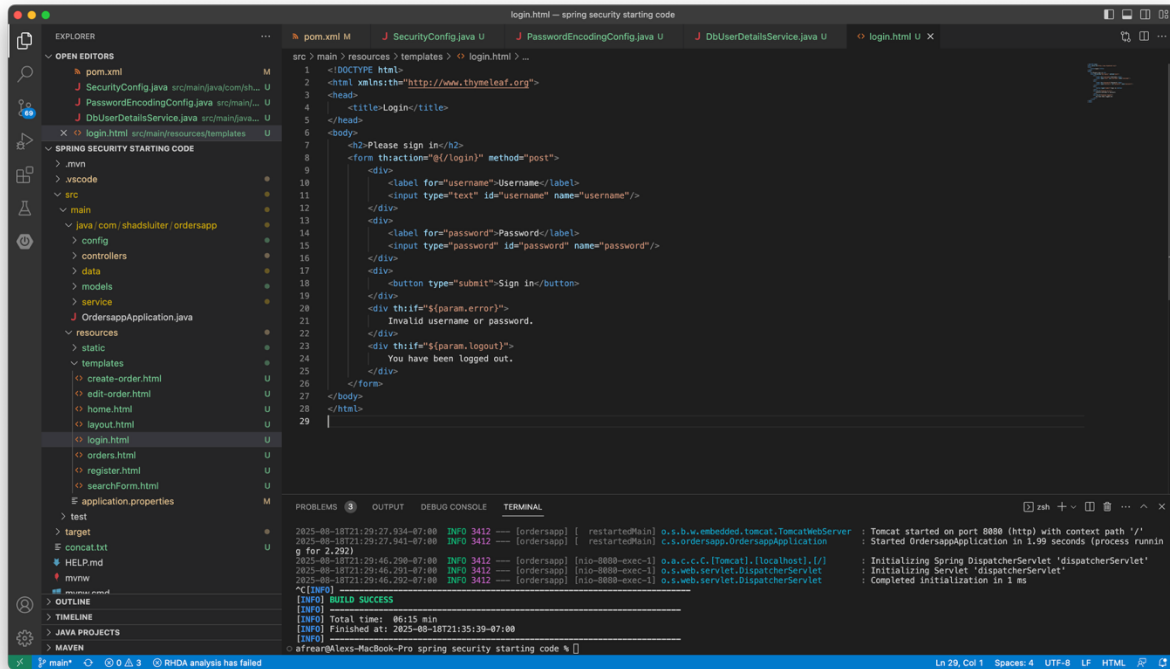


Figure 9 Custom `login.html` created in the templates folder, configured to post to `/login` and display error/logout messages.

2.2 – Update SecurityConfig to Use Custom Login

SecurityConfig Custom Login Update

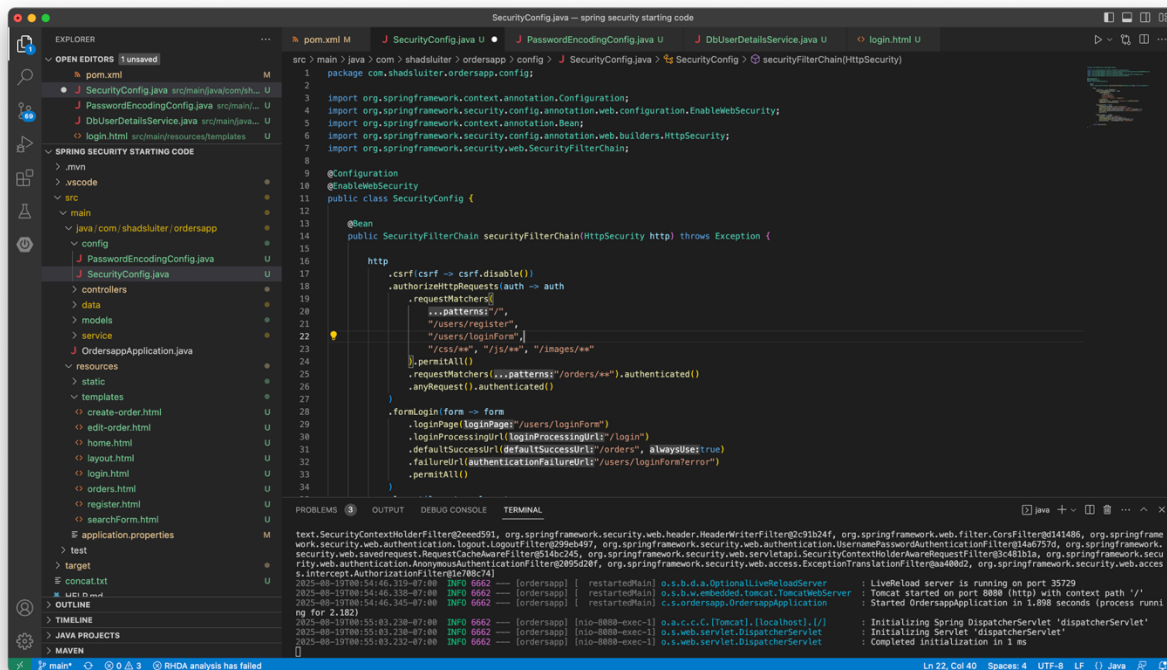


Figure 10 `SecurityConfig` updated to permit `/login` and to use `.loginPage("/login")` with `.failureUrl("/login?error")`

2.3 – Test the Custom Login Page

Login Page (Custom) Redirect

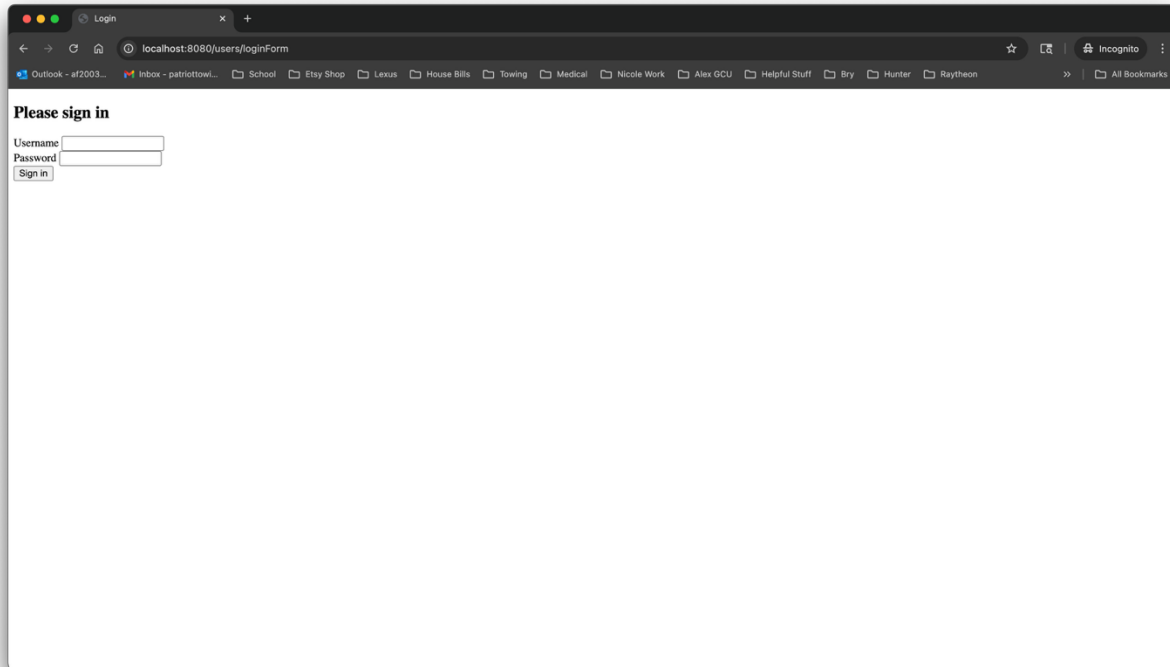
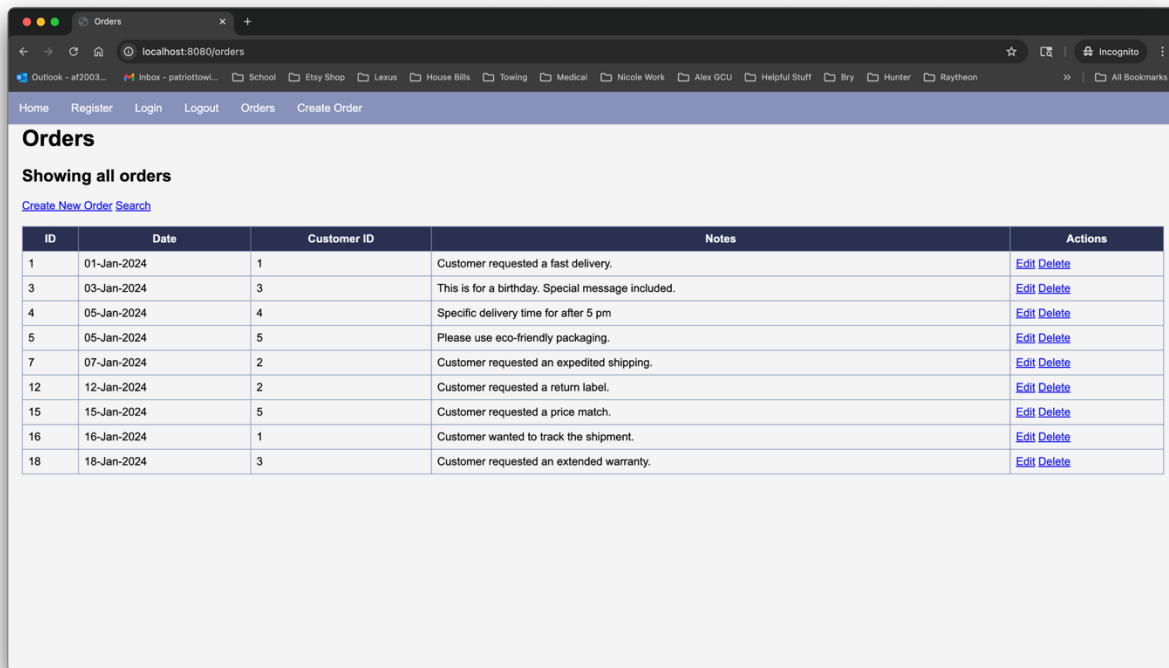


Figure 11 Navigating to /orders redirects to the custom /login page served from templates/login.html.

2.4 – Verify Access After Login

Orders Page After Login



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/orders'. The page has a navigation bar with links: Home, Register, Login, Logout, Orders, and Create Order. Below the navigation bar, the page title is 'Orders', followed by the text 'Showing all orders' and a link 'Create New Order Search'. A table of orders is displayed with the following data:

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	05-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	16-Jan-2024	3	Customer requested an extended warranty.	Edit Delete

Figure 12 After registering with BCrypt-encoded password and signing in, the protected `/orders` page is accessible.

Logout Implemented as POST

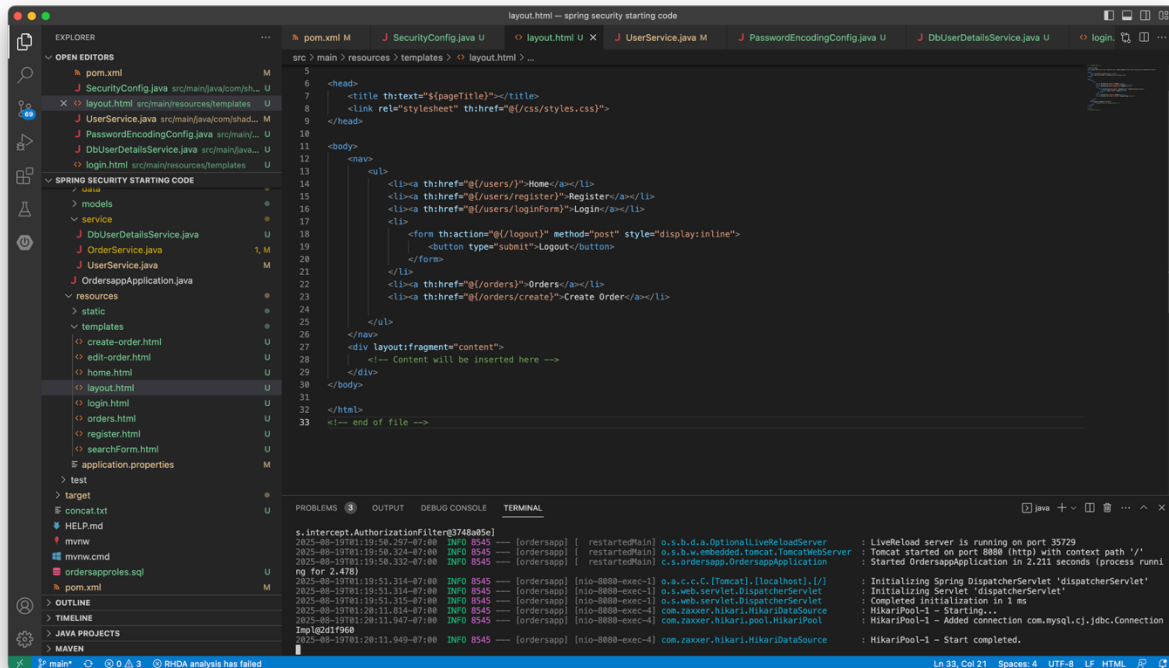
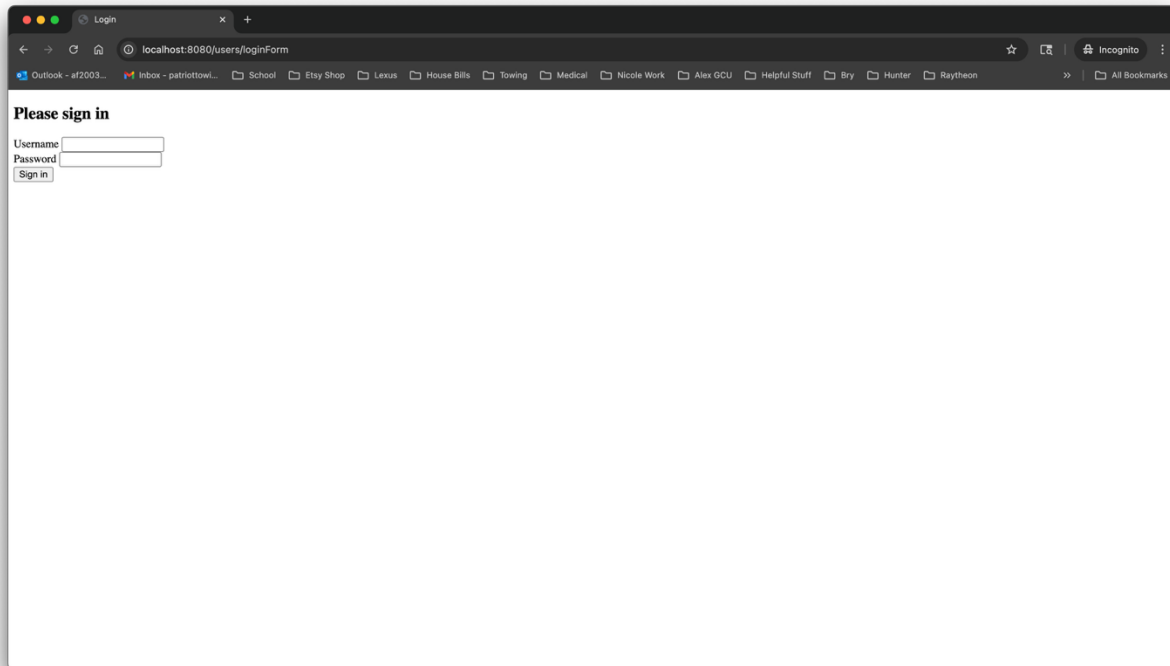


Figure 13 Navbar updated to submit a POST to `/logout`, matching Spring Security's expected logout method.

Logout and Redirect to Home



A screenshot of a web browser window. The address bar shows the URL `localhost:8080/users/loginForm`. The browser's bookmark bar is visible with various folders like 'Outlook', 'Inbox', 'School', 'Etsy Shop', 'Lexus', 'House Bills', 'Towing', 'Medical', 'Nicole Work', 'Alex GCU', 'Helpful Stuff', 'Bry', 'Hunter', and 'Raytheon'. The main content area of the browser displays a login form with the heading 'Please sign in'. Below the heading, there are two input fields: 'Username' and 'Password'. A 'Sign in' button is located below the 'Password' field.

Figure 14 After clicking Logout, the session is invalidated and the user is redirected; accessing `/orders` now prompts for login.

Part 3: Role-Based Access Control (RBAC)

3.1 – Add Roles to the User Entity

RBAC Rules in SecurityConfig

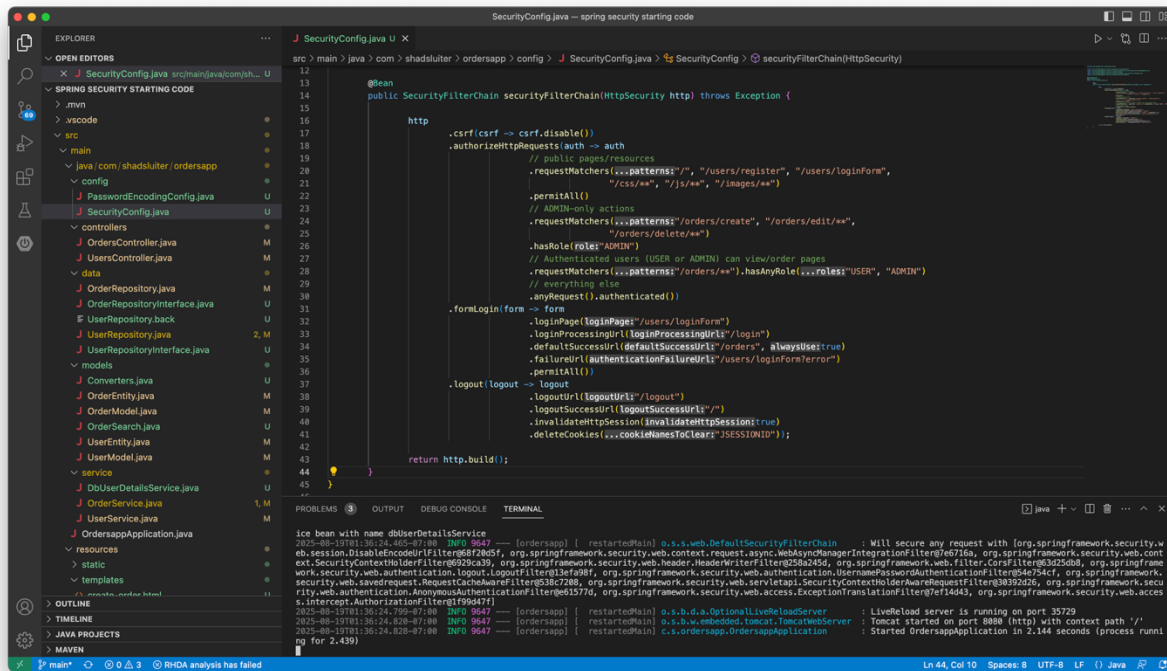


Figure 15 Security rules restrict `/orders/create|edit|delete` to `ROLE_ADMIN`, while `/orders/**` is available to authenticated users.

3.2 – Verify USER is denied on admin routes

USER Blocked from Admin Route

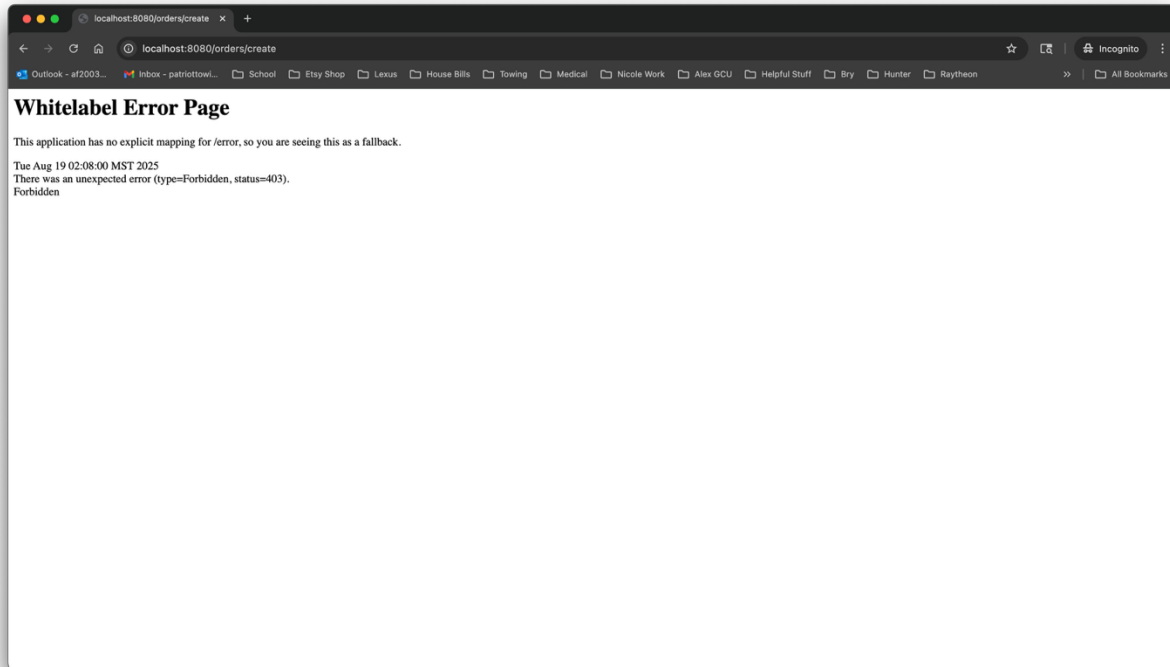


Figure 16 A ROLE_USER account attempts to open /orders/create and receives 403 Forbidden per RBAC rules.

3.3 – Verify ADMIN can access admin routes

Admin Can Access Create Order

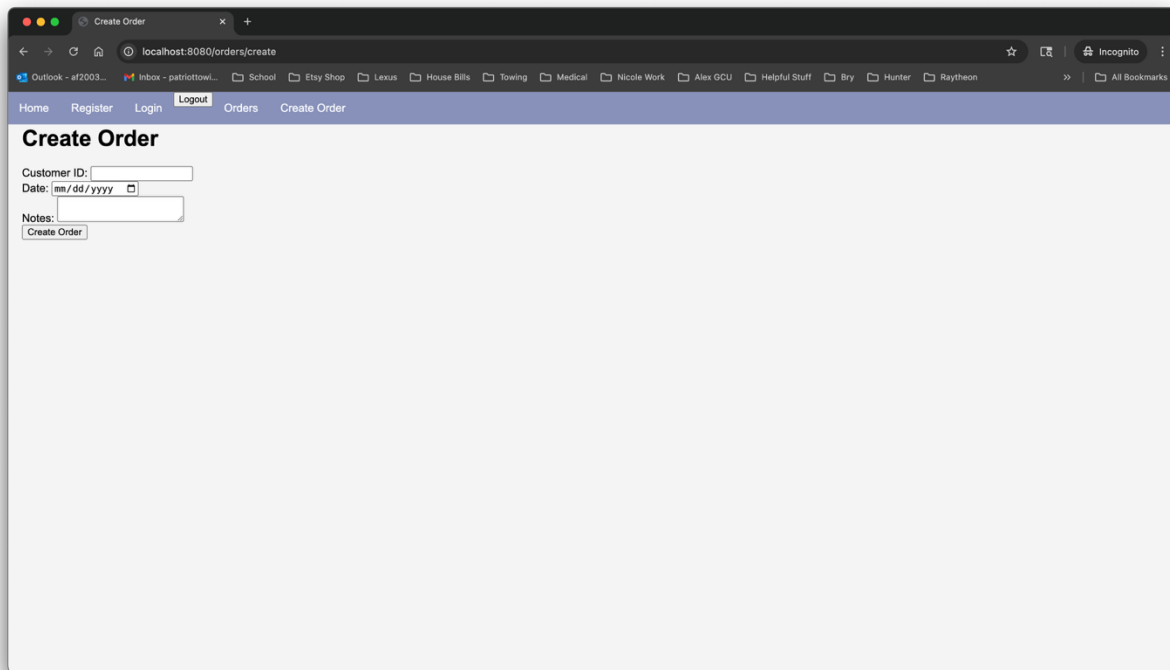


Figure 17 User with `ROLE_ADMIN` successfully accesses the `/orders/create` page.

3.4 – Show/Hide Navbar Items Based on Login & Role

Thymeleaf Security Extras Dependency

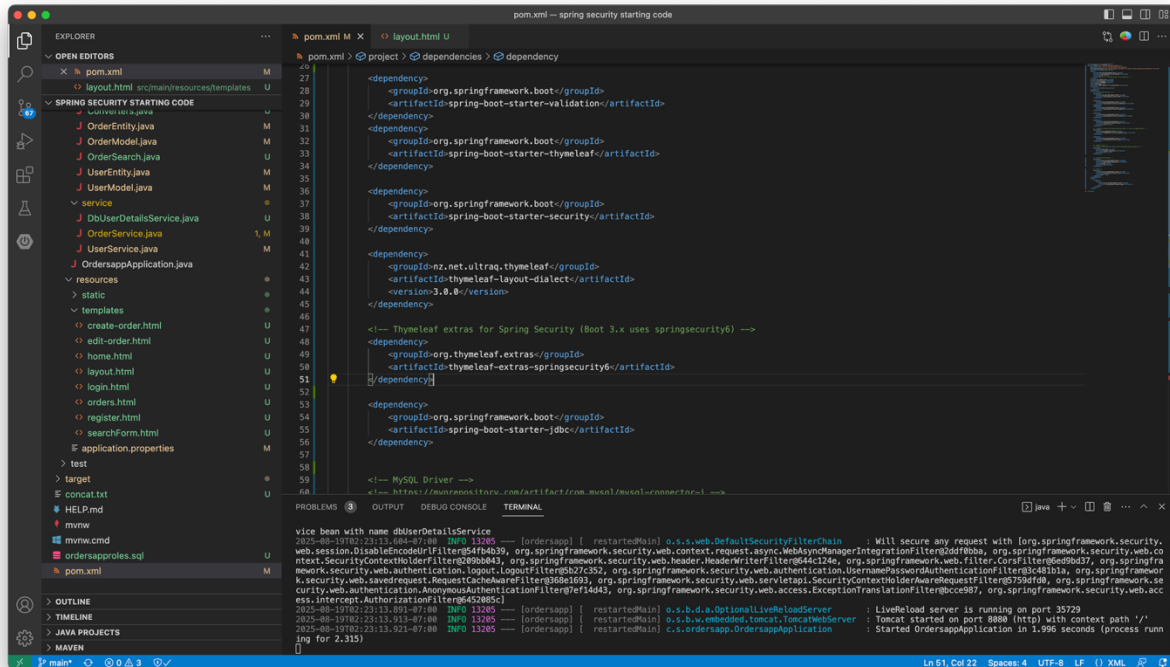


Figure 18 pom.xml updated with thymeleaf-extras-springsecurity6 to enable sec: * attributes.

Navbar (Logged Out)

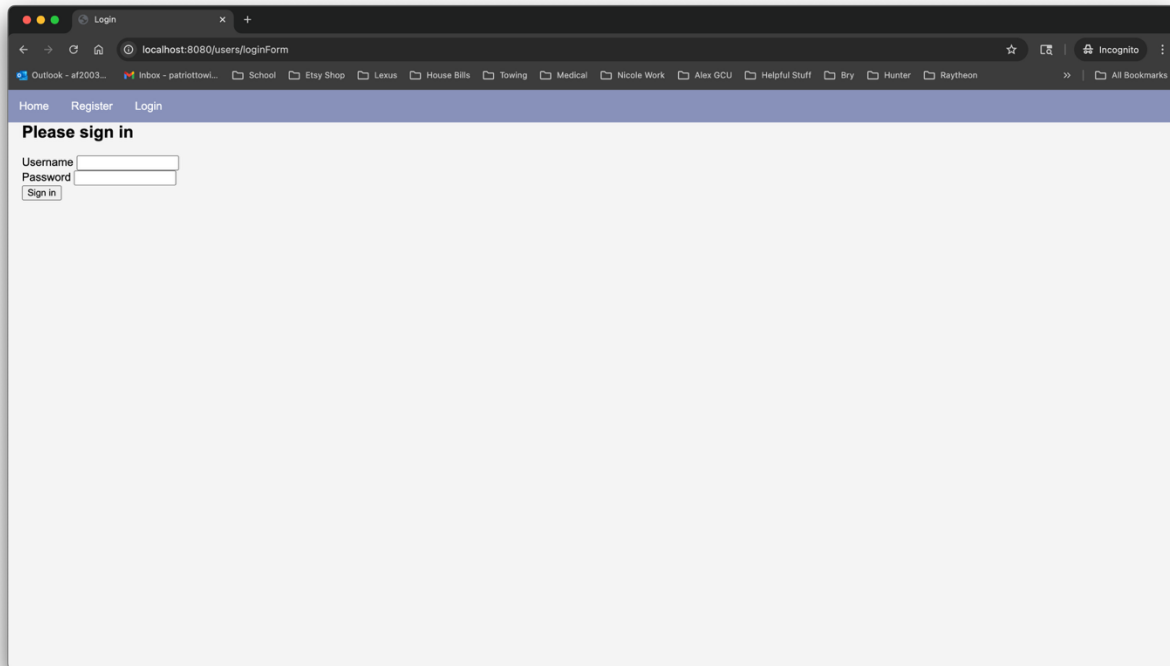


Figure 19 When not authenticated, only Home, Register, and Login are visible.

Navbar (Admin — Create Visible)

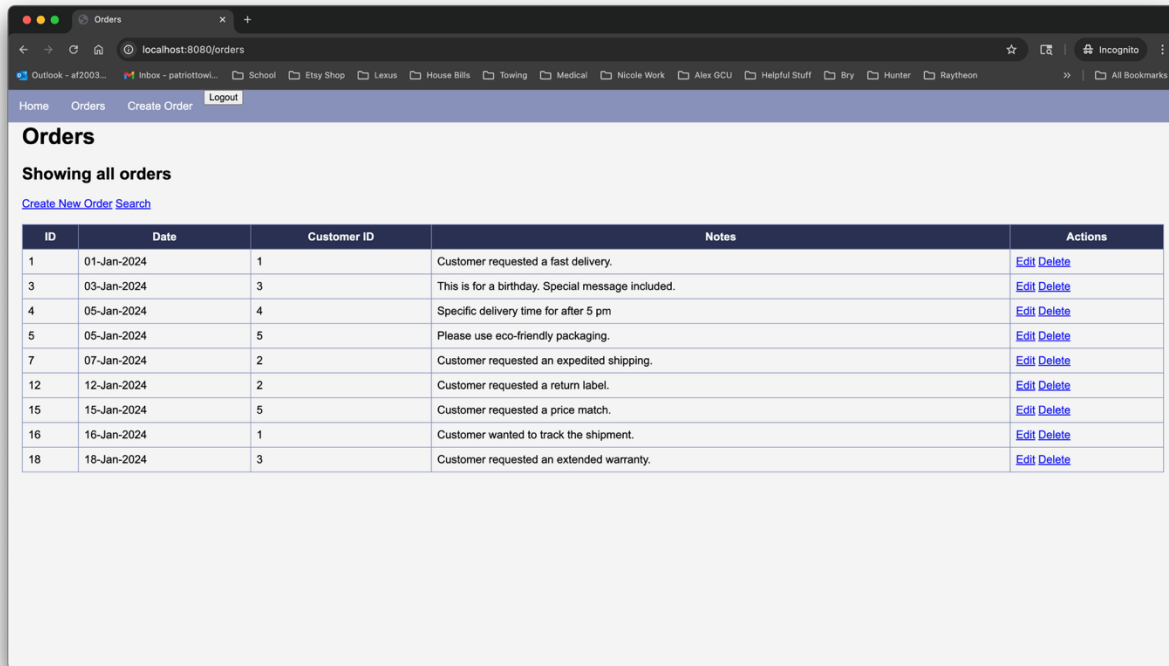


Figure 20 When logged in as `ROLE_ADMIN`, `Create Order` is visible along with `Orders` and `Logout`.

3.5 – Add Method-Level Security (defense in depth)

Enable Method-Level Security

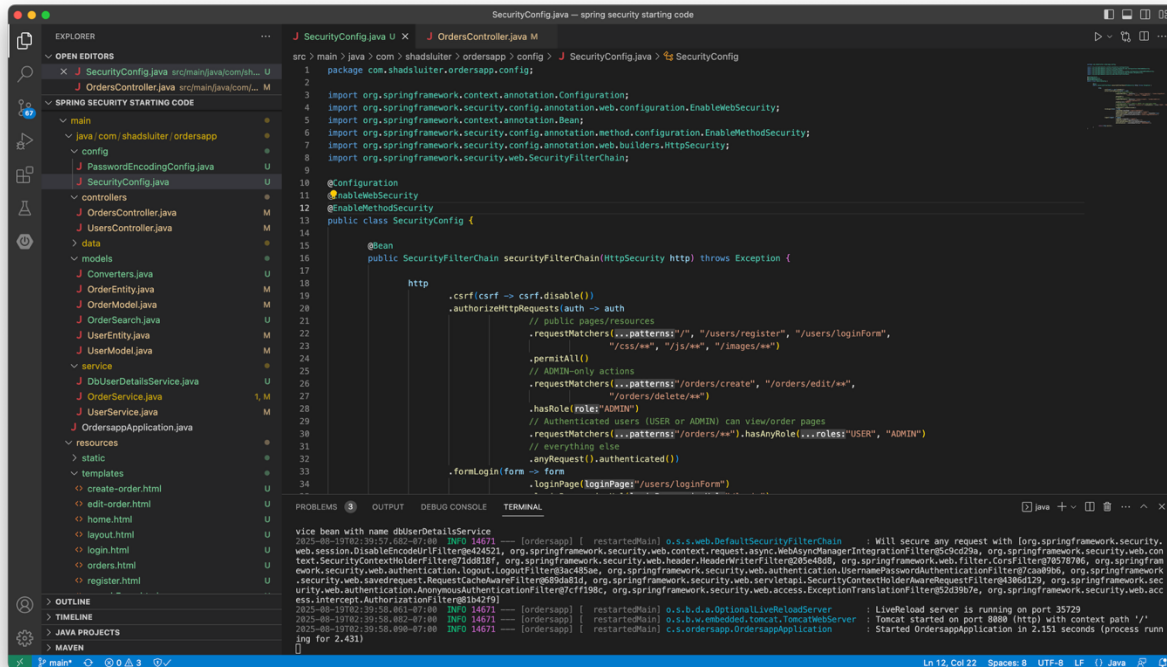


Figure 21 `SecurityConfig` updated with `@EnableMethodSecurity` to allow `@PreAuthorize` checks.

PreAuthorize on OrdersController (Admin Only)

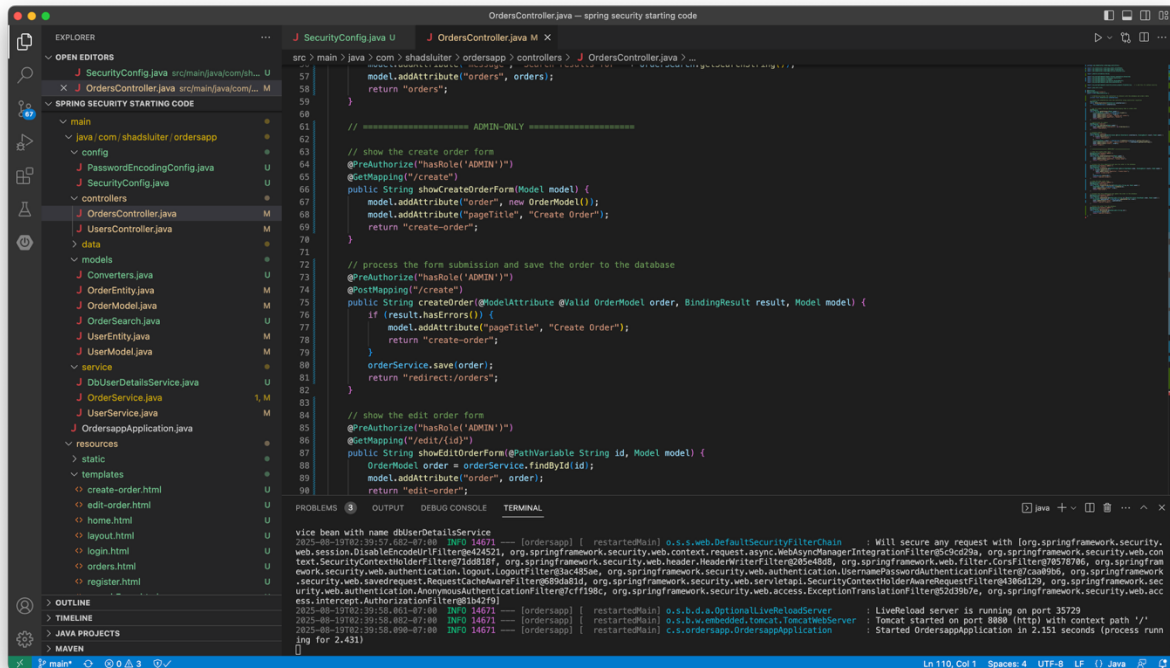
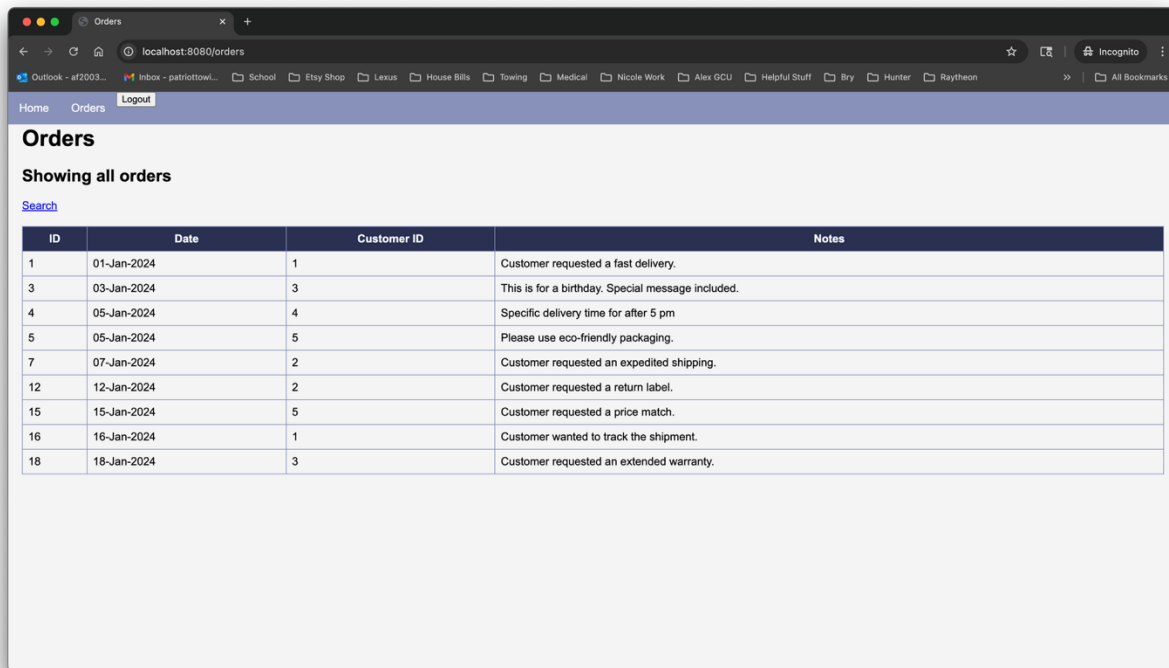


Figure 22 `@PreAuthorize('hasRole('ADMIN'))` applied to create/edit/delete endpoints in `OrdersController`.

3.6 – Hide admin actions in the UI (Edit/Delete/Create link)

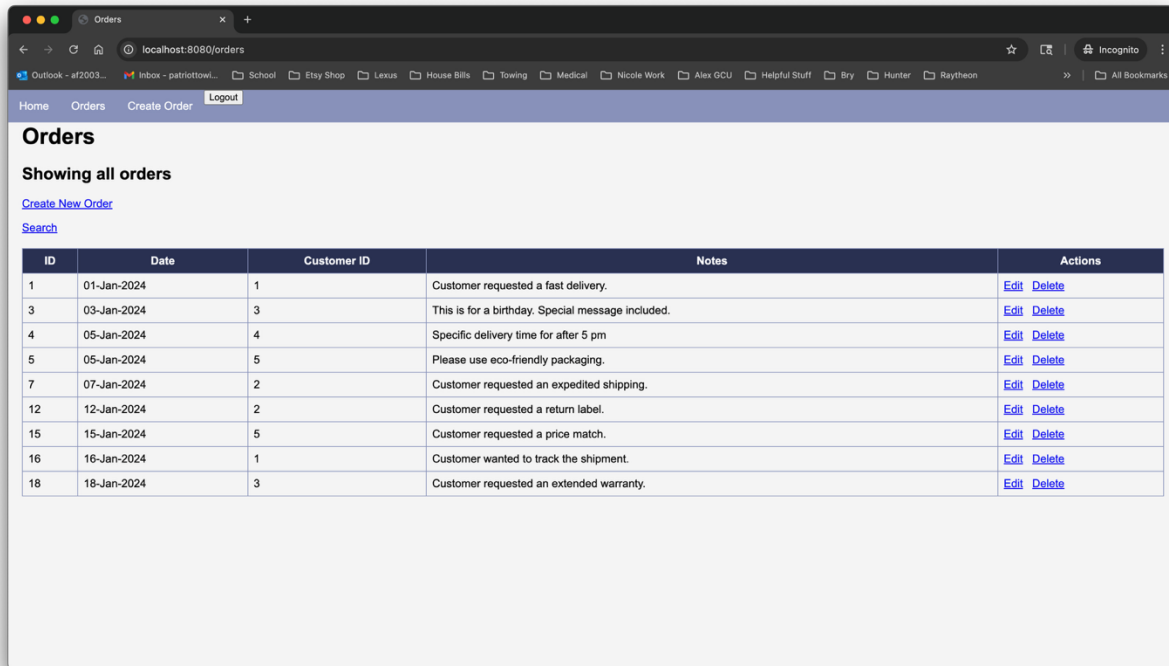
Orders Page Hides Admin Actions for USER



ID	Date	Customer ID	Notes
1	01-Jan-2024	1	Customer requested a fast delivery.
3	03-Jan-2024	3	This is for a birthday. Special message included.
4	05-Jan-2024	4	Specific delivery time for after 5 pm
5	05-Jan-2024	5	Please use eco-friendly packaging.
7	07-Jan-2024	2	Customer requested an expedited shipping.
12	12-Jan-2024	2	Customer requested a return label.
15	15-Jan-2024	5	Customer requested a price match.
16	16-Jan-2024	1	Customer wanted to track the shipment.
18	18-Jan-2024	3	Customer requested an extended warranty.

Figure 23 Logged in as `ROLE_USER`; the Orders page hides Create link and the Actions column.

Orders Page Shows Admin Actions



Orders

Showing all orders

[Create New Order](#)

[Search](#)

ID	Date	Customer ID	Notes	Actions
1	01-Jan-2024	1	Customer requested a fast delivery.	Edit Delete
3	03-Jan-2024	3	This is for a birthday. Special message included.	Edit Delete
4	05-Jan-2024	4	Specific delivery time for after 5 pm	Edit Delete
5	05-Jan-2024	5	Please use eco-friendly packaging.	Edit Delete
7	07-Jan-2024	2	Customer requested an expedited shipping.	Edit Delete
12	12-Jan-2024	2	Customer requested a return label.	Edit Delete
15	15-Jan-2024	5	Customer requested a price match.	Edit Delete
16	16-Jan-2024	1	Customer wanted to track the shipment.	Edit Delete
18	18-Jan-2024	3	Customer requested an extended warranty.	Edit Delete

Figure 24 Logged in as `ROLE_ADMIN`; the Orders page shows Create, Edit, and Delete options.

3.7 – Add a Custom “Access Denied” (403) Page

Custom Access Denied Page

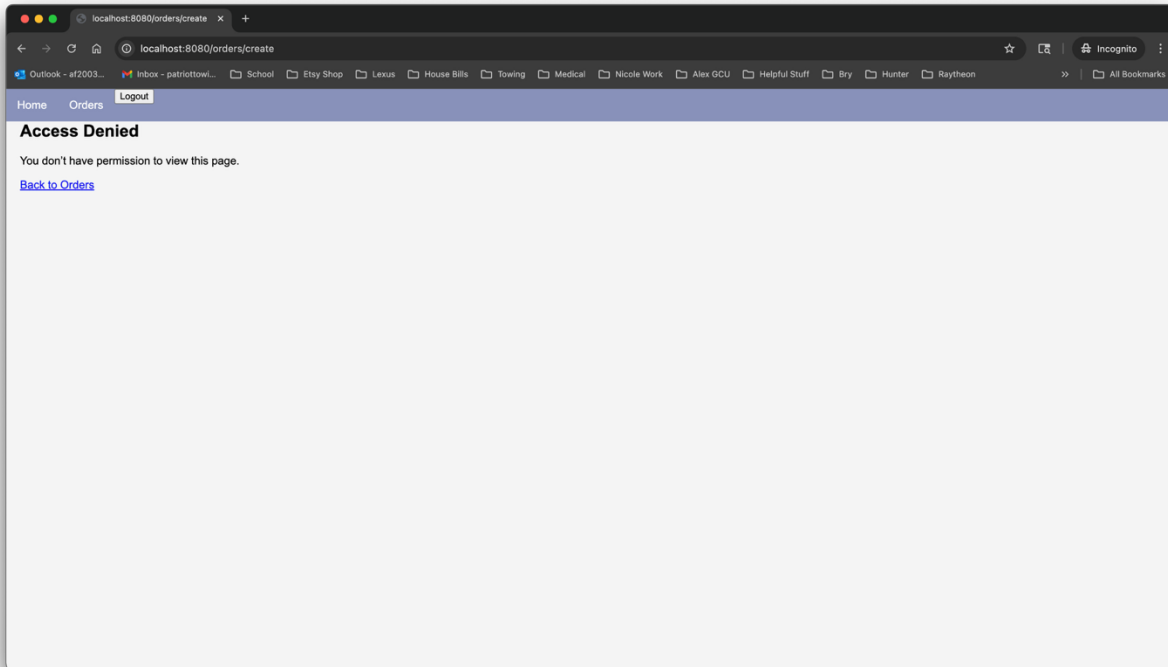


Figure 25 /orders/create denied to ROLE_USER shows the custom 403 page via `exceptionHandling().accessDeniedPage('/access-denied')`.

Part 4 – Summary of Key Concepts

When I was working through this activity, I realized that Spring Security relies on a few key classes to handle how authentication and authorization work. For example, the `SecurityFilterChain` is where the rules are defined for which endpoints need authentication and what roles are allowed to access them. The `AuthenticationManager` is the piece that processes login attempts and checks if the credentials are valid, while the `UserDetailsService` is responsible for pulling in user data like usernames, passwords, and roles from either memory or a database. I also learned how important the `PasswordEncoder` is because it ensures that passwords are stored securely as hashes instead of plain text. All of this gets tied together with the `HttpSecurity` class, which gives us a way to configure web-based security in a clean, structured way.

Another big concept that stood out to me is how the filter chain works in Spring Security. Every request that comes into the application goes through this chain before it ever reaches a controller. Each filter has its own job, like checking if a user is authenticated, making sure the user has the right authorization, or applying protections like CSRF. If a request doesn't meet the requirements, it's blocked right there with either a 401 Unauthorized or a 403 Forbidden, depending on the issue. Only if the request passes all the filters does it actually move on to the controller. I think this setup makes sense because it keeps the controller focused on business logic, while the filter chain handles all the security in one place.